

CYBER WINTER

- WEEK 1 – INTRODUCTION TO CYBERSECURITY AND ETHICAL HACKING WITH SYLLABUS DISTRIBUTION
- WEEK 2 – NETWORKING FUNDAMENTALS FOR ETHICAL HACKING
- WEEK 3 – INTRODUCTION TO FOOTPRINTING AND RECONNAISSANCE
- WEEK 4 – SCANNING AND ENUMERATION TECHNIQUES
- WEEK 5 – SYSTEM AND NETWORK EXPLOITATION BASICS WITH WIRELESS ATTACKS
- WEEK 6 – WEB APPLICATION SECURITY FUNDAMENTALS , REMEDIATIONS AND REPORTING

CYBERSECURITY

Cybersecurity is the practice ofprotectingsystems, networks, and applications from digital attacks. In this rapidly growing field, therearediverseroles thatcater to different skill sets and interests.

Key Roles in Cybersecurity

01 **Security Analyst:** They generally monitors, analyzes and investigates security systems to detect and respond to potential threats to ensures systems are protected.

02 **Penetration Tester (Pentester/Ethical Hacker):** They Simulates real-world attacks to identify and fix vulnerabilities in systems and applications. This is one of the most exciting roles for those interested in hacking legally! It's about being one step ahead of attackers to ensure systems are secure. **SOC Analyst (Security Operations Center Analyst):**

03 Works in a SOC team is to detect and respond to cybersecurity incidents. Uses tools like SIEM (Security Information and Event Management) to monitor network activities.

04 **Incident Responder:** They generally handles security breaches and minimizes damage during cyber incidents and works to restore normal operations and improve defenses.

05 **Forensics Expert:** They Investigates cybercrimes and gathers digital evidence and works closely with law enforcement to solve cases and prevent future breaches.

Note: Cybersecurity jobs are expected to grow by 35% by 2030. Organizations are looking for skilled professionals to protect their digital assets and with dedication and practice, you could become a key part of this growing field.

PENETRATION TESTING / PENTESTING

Pentesting is the process ofsimulatingcyberattackstoidentifyvulnerabilitiesinsystems, networks, applications, and APIs.Itis donetofindandfixweaknessesbeforemaliciousHackerscan exploit them.



Types of Penetration Testing

01 **Web Application PT:** In this the major focus are Websites, APIs and Web-based applications like testing for vulnerabilities like SQL Injection or XSS.

02 **Network PT:** In this the major focus are Routers, firewalls, servers, and other network devices like Identifying open ports that could allow unauthorized access.

03 **API PT:** In this the major focus are Application Programming Interfaces (APIs) used for communication between services like testing for insecure authentication, excessive data exposure, or improper error handling. **Mobile Application PT:** In this the major focus are Android and iOS apps like testing mobile apps for insecure data storage or API weaknesses.

04

05 **Cloud PT:** In this the major focus are Cloud services like AWS, Azure, and Google Cloud like ensuring cloud environments are configured securely.

Note: Pentesting includes all steps as in Ethical Hacking like Reconnaissance, Scanning, Exploitation, Reporting, Remediation Testing and the tools included Burp Suite, SQLmap, Postman, OWASP ZAP, Nmap and much more.

WHAT IS WEB APPLICATION?

A **Web application** is a softwareapplication that runs on awebserverandisaccessed by users through a web browser over the Internet. Unlike traditional desktop applications that require installation on individual devices, web applications are platform-independent and can be used on any device with a compatible browser and network connection.



Characteristics of Web Applications

01 **Browser-Based:** Accessible via web browsers like Chrome, Firefox, Safari, or Edge.

02 **Client-Server Model:**

- **Client**: The front-end user interface running in the browser
- **Server**: The back-end that processes requests, retrieves data, and sends responses.

03 **Network Dependency:** Requires an active internet or intranet connection, though some may offer offline capabilities.

04 **Interactive and Dynamic:** Provides real-time interactivity (e.g., forms, buttons, data updates) often using JavaScript or frameworks like React, Angular, or Vue.js.

05 **Updates and Maintenance:** Centrally managed, so updates and fixes are applied on the server without requiring user-side changes.

WEB APPLICATION

Types of Web Applications

- **Static Web App:** Deliver pre-rendered content without dynamic interaction. Examples: Personal portfolios, basic blogs.
- **Dynamic Web App:** Provide real-time interactivity and content updates. Examples: Social media platforms, e-commerce sites.
- **Single-Page Applications (SPAs):** Operate within a single browser page, dynamically updating content without reloading the page. Examples: Gmail, Google Maps. **Progressive Web App (PWAs):** Combine the capabilities of web and native apps, offering
- offline access and enhanced performance. Examples: Spotify Web, Twitter Lite. **Content Management Systems (CMS):** Allow users to create, manage, and modify content without coding. Examples: WordPress, Joomla. **E-commerce Applications:**
- Facilitate online shopping and transactions. Examples: Amazon, eBay. **Portal Web Applications:** Provide a gateway to various related services and functionalities. Examples: Government portals, university websites.
-
-

Technologies Used in Web Applications

- **Front-End (User Interface):** HTML, CSS, JavaScript. **Frameworks/Libraries:** React, Angular, Vue.js **Back-End (Server-Side**
- **Logic): Programming Languages:** Python, Java, PHP, Node.js. **Frameworks:** Django, Flask, Express.js **Database: Relational:**
- MySQL, PostgreSQL. **Non-relational:** MongoDB, Firebase **Web Servers:** Apache, Nginx **Protocols:** HTTP/HTTPS, WebSocket (for
- real-time communication)
-

INTRODUCTION TO HTTP BASICS

HTTP (Hypertext TransferProtocol) is thefoundationofcommunication on the web. It facilitates the exchange of data between a client (browser)and a server which includesRequests (from client) and Responses (from server). It is the backbone of web applications and APIs and understanding how it works helps you analyze vulnerabilities in requests and responses.

Understanding HTTP Requests

1. **Request Line:** Contains the HTTP method (e.g., GET, POST), URL, and HTTP version.
    a. **Common HTTP Methods**:
        i. **GET**: Retrieves data from the server (e.g., webpage content).
        ii. **POST**: Sends data to the server (e.g., submitting forms).
        iii. **PUT**: Updates or creates a resource.
        iv. **DELETE**: Removes a resource.
        v. **HEAD**: Retrieves headers without the body.
2. **Headers**: Provide metadata about the request (e.g., User-Agent, Host).
3. **Body**: It includes data like form submissions or file uploads.

Understanding HTTP Responses

1. **Status Line:** Includes the HTTP version, status code, and status message.
    a. **HTTP Status Codes**:
        i. **1xx (Informational Responses):** 100 Continue, 101 Switching Protocols, etc.
        ii. **2xx (Success)**: 200 OK, 201 Created, etc.
        iii. **3xx (Redirection Responses):** 301 Moved Permanently, 302 Found, 304 Not Modified, etc.
        iv. **4xx (Client Error):** 400 Bad Request, 401 Unauthorized, 404 Not Found, etc.
        v. **5xx (Server Error)**: 500 Internal Server Error, 503 Service Unavailable, etc.
2. **Headers**: Provide information like content type, caching, and security details.
3. **Body**: It contains the requested data (e.g., HTML, JSON).

HTTP HEADERS (REQUESTS)

General Headers

- **Accept:** Specifies themedia types the client is willing to accept. Example: Accept:text/html, application/json
- **Accept-Charset:** Indicates the character sets the client supports. Example: Accept-Charset: utf-8 **Accept-Encoding:** Specifies
- the encoding algorithms (e.g., gzip) the client supports. Example: Accept-Encoding: gzip, deflate **Accept-Language:** Specifies
- the preferred languages for the response. Example: Accept-Language: en-US, en;q=0.9 **Authorization:** Credentials for
- authentication. Example: Authorization: Basic <credentials> **Cache-Control:** Directives for caching mechanisms. Example:
- Cache-Control: no-cache **Connection:** Options for the connection. Example: Connection: keep-alive **Content-Length:** Size of
- the request body in bytes. Example: Content-Length: 348 **Content-Type:** Media type of the request body. Example: Content-
- Type: application/json **Cookie:** Sends stored cookies to the server. Example: Cookie: sessionId=abc123 **Host:** Specifies the
- target host and port number. Example: Host: example.com **Referer:** The URL of the resource from which the request
- originated. Example: Referer: https://example.com **User-Agent:** Information about the client software. Example: User-Agent:
- Mozilla/5.0
-
-

Security Headers in Requests

- **Origin:** Indicates the origin of the request, used in CORS. Example: Origin: https://example.com
- **Upgrade-Insecure-Requests:** Signals preference for HTTPS. Example: Upgrade-Insecure-Requests: 1
- **X-CSRF-Token:** Anti-CSRF tokens to prevent CSRF attacks. Example: X-CSRF-Token: abc123

HTTP HEADERS (RESPONSES)

General Headers

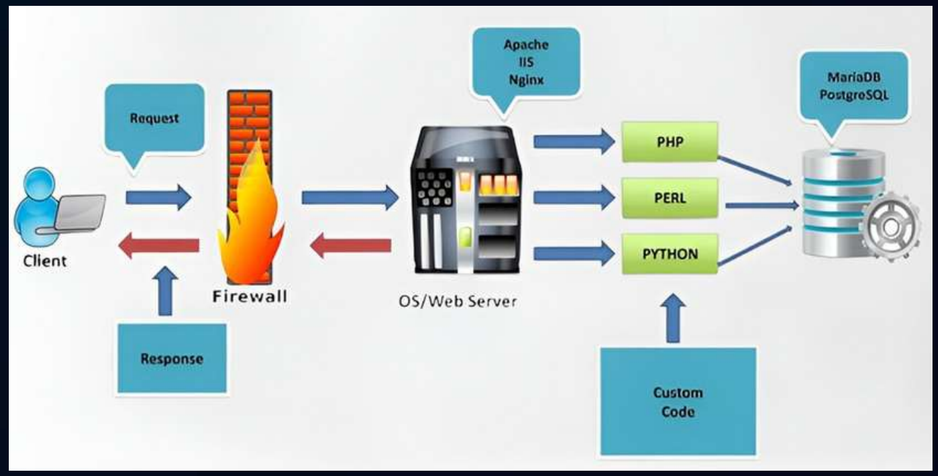
- **Content-Type:** Media typeof the response. Example: Content-Type:application/json
- **Content-Length:** Size of the response body in bytes. Example: Content-Length: 128 **Content-Encoding:**
- Indicates how the response is encoded. Example: Content-Encoding: gzip **Date:** The date and time the
- response was generated. Example: Date: Fri, 10 Jan 2025 12:00:00 GMT **Server:** Information about the
- server. Example: Server: Apache/2.4.41 **Cache-Control:** Directives for caching mechanisms. Example:
- Cache-Control: no-store **Set-Cookie:** Sets a cookie in the user's browser. Example: Set-Cookie:
- sessionId=abc123; HttpOnly; Secure **ETag:** A unique identifier for a specific version of a resource. Example:
- ETag: "12345"

Security Headers in Requests

- **HSTS:** EnforcesHTTPS for allfuture requests. Example: Strict-Transport-Security: max-age=31536000; includeSubDomains
- **CSP:** Restrictssources of contentto mitigate XSS attacks.Example:Content-Security-Policy:default-src 'self'
- **X-Content-Type-Options:** Prevents browsers from interpreting files as a different MIME type. Example: X-Content-Type-Options: nosniff
- **X-Frame-Options:** Prevents embedding the page in aniframe to mitigateclickjacking. Example: X-Frame-Options:DENY
- **X-XSS-Protection:** Enables or disables the browser's built-in XSS protection. Example: X-XSS-Protection: 1; mode=block
- **Referrer-Policy:** Controls the amount of referrer information included with requests. Example: Referrer-Policy: no-referrer
- **Expect-CT:** Encourages the use of Certificate Transparency logs. Example: Expect-CT: max-age=86400
- **Access-Control-Allow-Origin:** Specifies domains allowed to access the resource, used in CORS.
- **Access-Control-Allow-Methods:** Specifies allowed HTTP methods for CORS.
- **Access-Control-Allow-Headers:** Specifies allowed headers in the request for CORS. Example: Access-Control-Allow-Headers: Content-Type, Authorization

Introduction to Web Application Security:

Web application security is about protecting websites and online apps from threats and unauthorized access. It involves measures to keep user data safe, prevent attacks like SQL injection or Cross-Site Scripting (XSS), control access, and encrypt data. The goal is to keep user information private, prevent cyber intrusions, and ensure that the web application functions smoothly and securely.



But for checking and testing the security of Web application, APIs, Network, AI, Mobile and Cloud, we are having an Open Source Platform named as OWASP.

OWASP:

OWASP (Open Web Application Security Project) is a global group that helps improve web application security by offering free tools and guidelines to protect websites from threats. It offers a list of the 10 biggest security risks for web applications, APIs, Network, AI, Mobile and cloud that helps organizations know what to protect against. It's updated regularly to address new threats.

THE LATEST OWASP TOP 10 VULNERABILITIES

01 BrokenAccess Control

02 Cryptographic Failures

03 Injection Insecure Design

04 Security Misconfiguration

05 Vulnerable and Outdated Components

06 Identification and Authentication Failures

07 Software and Data Integrity Failures

08 Security Logging and Monitoring Failures

09 Server-Side Request Forgery (SSRF)

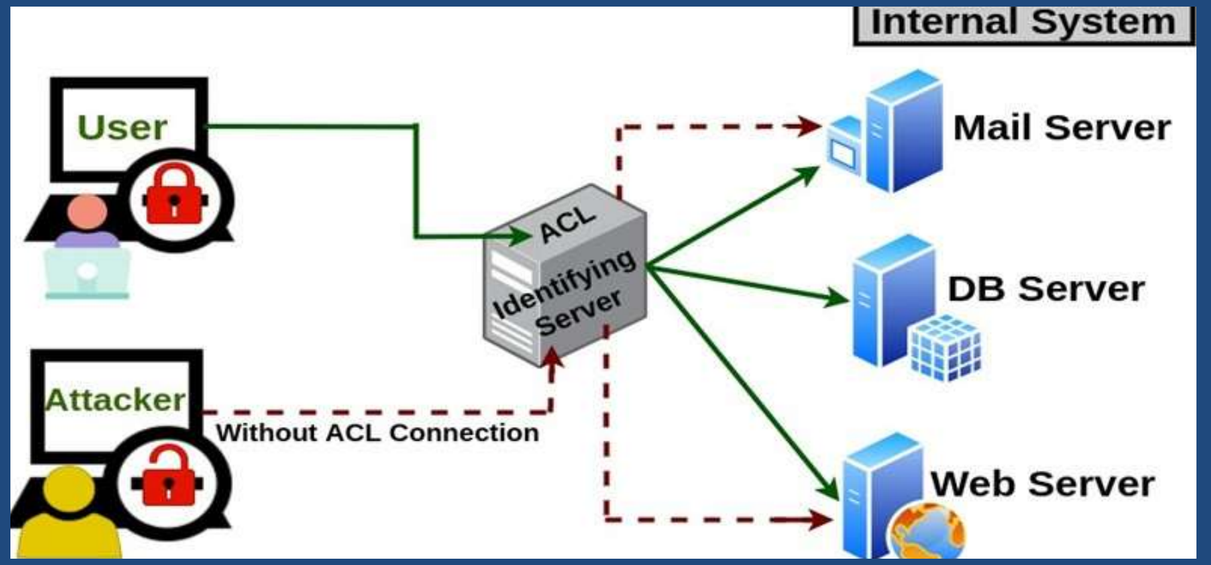
10



OWASP TOP 10

Broken Access Control

Access control ensures users canonly perform actions oraccess resources they are authorized for.
Broken access control occurs when these restrictions fail. In simple terms, it is a security vulnerability that allows unauthorized users to access, modify, or delete data they shouldn't have access to.



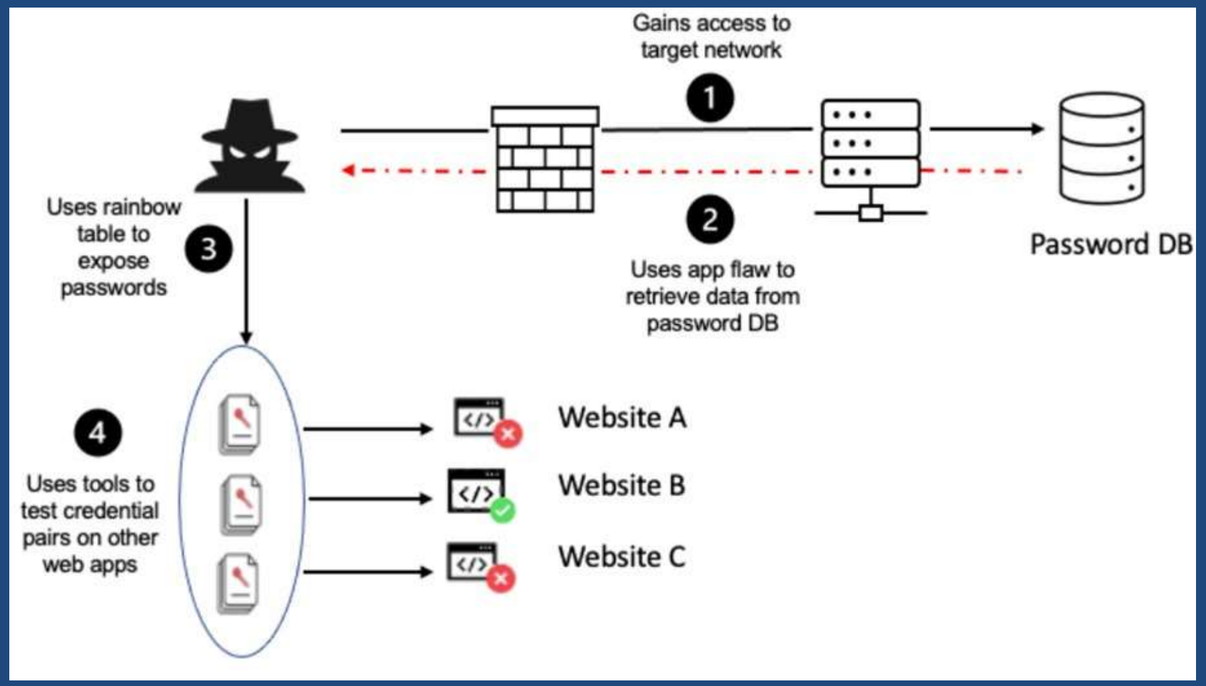
Example:

For example, if a person changes the web address (URL), they might be able to view someone else's account if the website doesn't check if they're allowed to see it. To prevent this, websites need to make sure each user can only access their own information.

OWASP TOP 10

Cryptographic Failures

It refers to mistakes in protecting sensitivedata,likeusing**weak encryption** or failing to encrypt data at all. This makes data vulnerable to hackers.



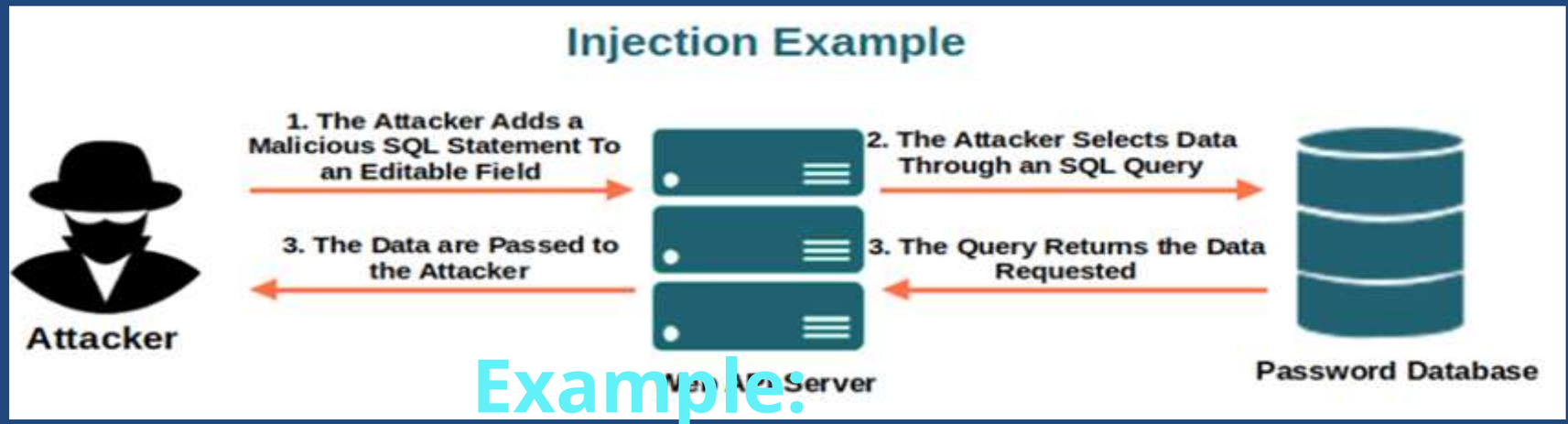
Example:

- Usingoutdated cryptographic algorithms (e.g., MD5, SHA-1). Storing sensitive
- data like passwords without proper hashing or encryption.

OWASP TOP 10

Injection

Injection is a security problem wherehackersput harmful code into a website to make it do things it shouldn’t, like giving out private data. This happens when the website doesn’t check user input properly.



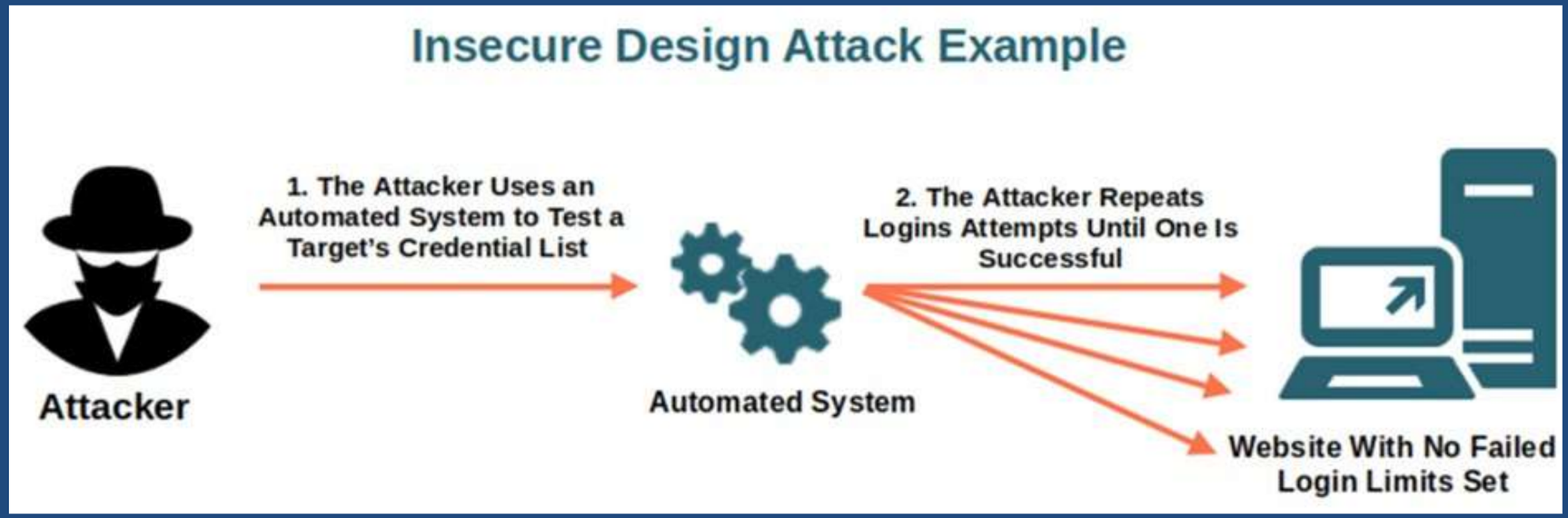
Example:

Ifawebsiteallowsharmful code instead of just names, a hacker could steal information. Common types of attacks include SQL Injection (to access data) = **admin‘ or ‘1’ = ‘1** and Command Injection (to control the server). For mitigating this Vulnerability we can use parameterized queries or prepared statements. Also we can validate and sanitize all user inputs.

OWASP TOP 10

Insecure Design

It means **weak security** choices madeduringthe planningand building of a website or app. This can lead to vulnerabilities that hackers can exploit.



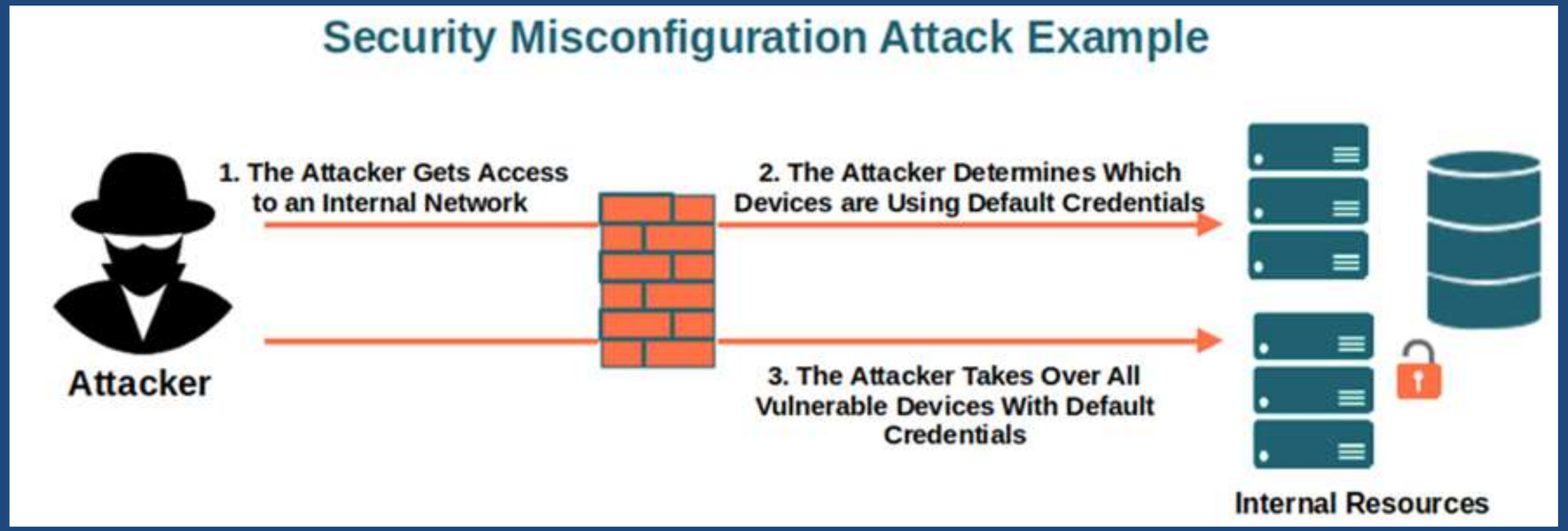
Example:

- An ATMthat doesn't require a PIN is insecure by design. To prevent this, security needs to be considered from the start of the design process. Not implementing proper session timeouts.
- Designing applications without threat modeling.
-

OWASP TOP 10

Security Misconfiguration

It means **making** **mistakes** when **setting upthe security** fora websiteor app. If these settings are wrong, it can leave the system open to attacks, even if the app itself is built well.



Example:

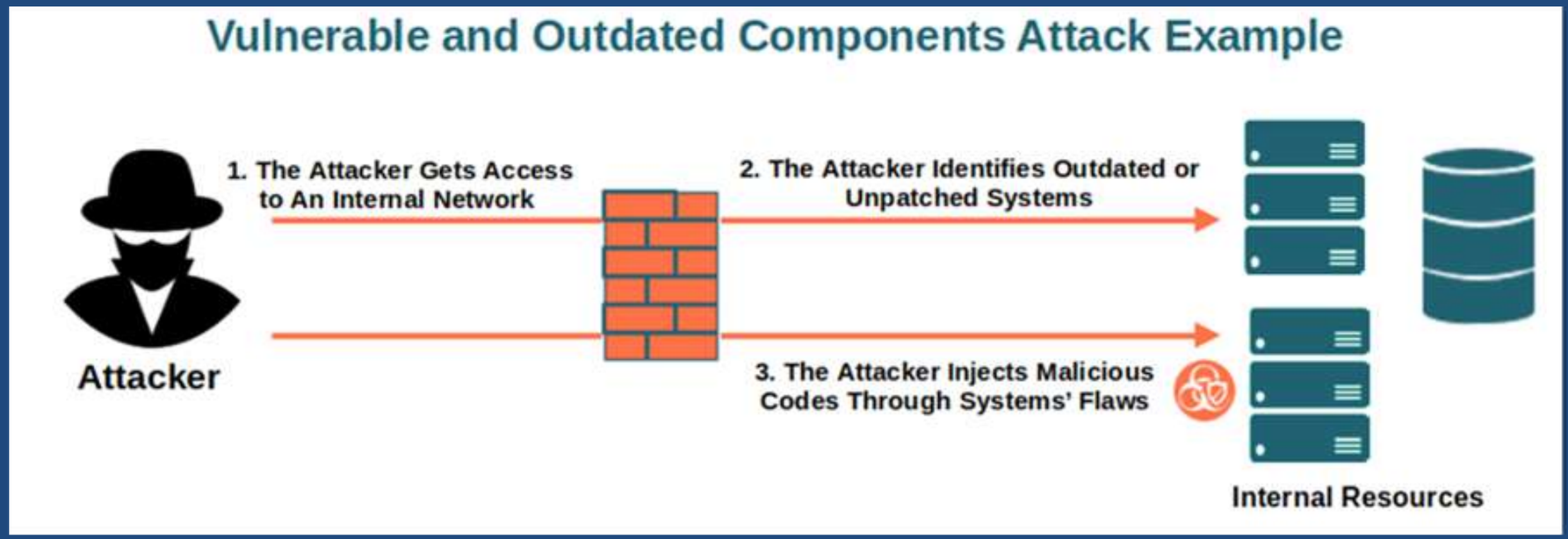
- Leaving defaultaccounts active with default passwords.
- Exposing sensitive files such as .env or config.yml.

Prevention: Automate security configuration checks also remove unnecessary features or files and do regularly patch and update all software.

OWASP TOP 10

Vulnerable and Outdated Components

Itmeans usingoldor unsafeparts in a websiteor appthathaveknownsecurity problems. If not updated, hackers can use these weaknesses to attack the system.



Example:

Anoldlockonadoor is easy to pick if it’s not upgraded, leaving the house at risk.

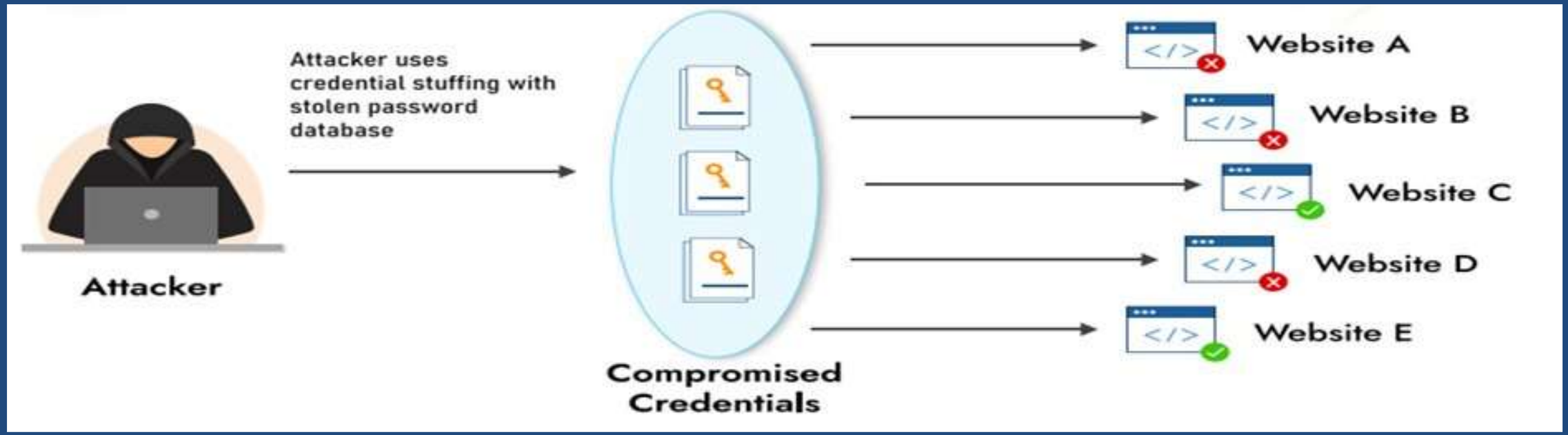
OWASP TOP 10

Identification and Authentication Failures

Ithappens whenawebsitedoesn’tproperlyconfirmwhoauseris,allowinghackers easier access to accounts.

Example:

weakpasswords,no automatic logouts, and no two-factor authentication.



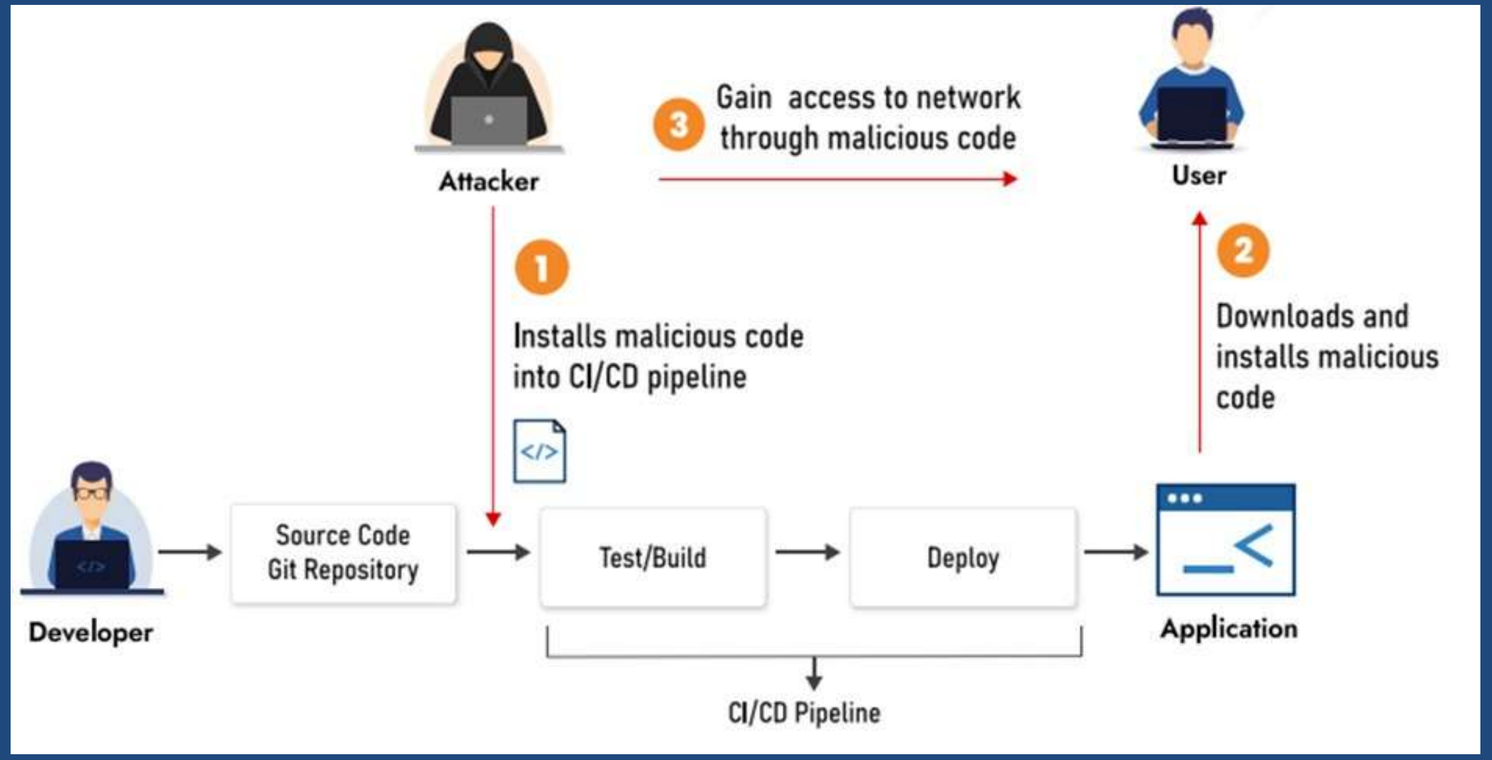
OWASP TOP 10

Software and Data Integrity Failures

When an appdoesn’tprotect its software ordatafrombeingtampered with. This can lead to security risks, like adding harmful code or changing important data without permission.

Example:

Malwareorharmful code, Data corruption



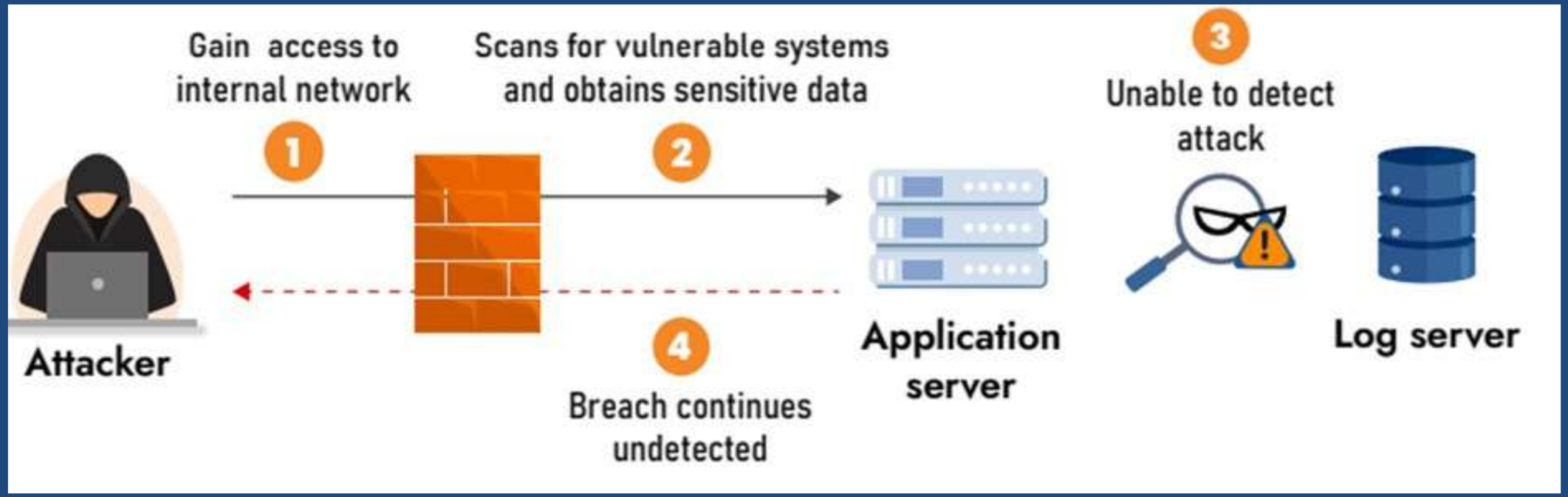
OWASP TOP 10

Security Logging and Monitoring Failures

Thismeansan app or website doesn’thave good trackingoralert systems, so security problems are hard to detect and respond to. Without proper logging and monitoring, attacks can happen without anyone noticing, and issues might not get fixed in time.

Example:

Repeatedpassword guesses might go unnoticed without tracking.



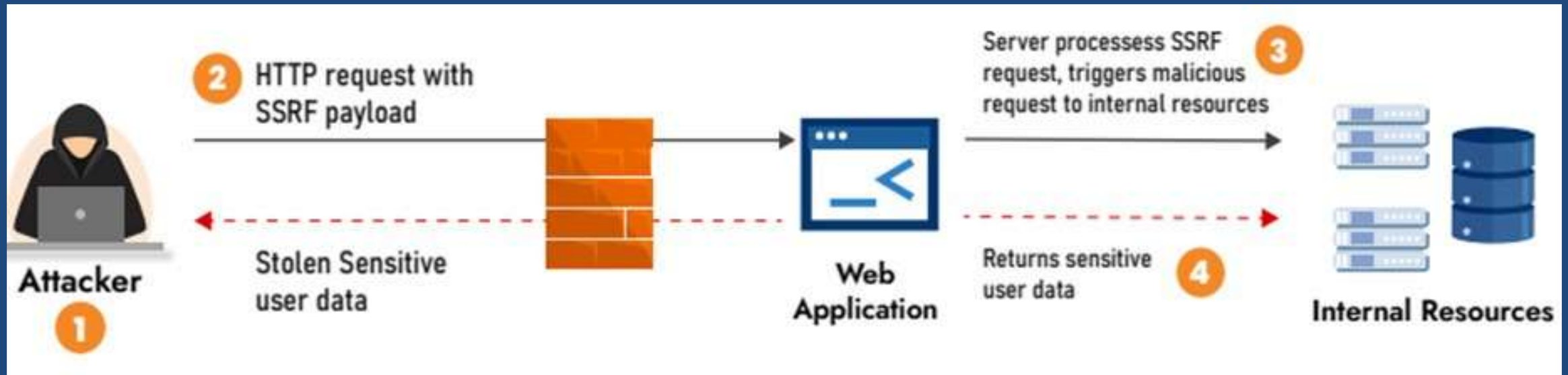
OWASP TOP 10

Server-Side Request Forgery

Server-Side Request Forgery (SSRF) is a security vulnerability where an attacker tricks a server into making unauthorized requests to systems the attacker couldn't normally access.

Example:

Imagine you can ask a librarian to access certain books in a restricted area of a library, even if you're not allowed in yourself. You tell the librarian to get a specific book, and they do it, trusting you. Similarly, in SSRF, the hacker tricks the server into accessing restricted areas or data that the hacker couldn't access on their own.



COMMON ATTACKS AGAINST WEB APPLICATIONS

01 Brute force

02 Credential stuffing

03 SQL injection & form jacking injections

04 Cross-site scripting

05 Cookie poisoning

06 Man-in-the-middle (MITM)

07 Sensitive data disclosure

08 Insecure deserialization

09 Session hijacking



Common Web Application Security Mistakes:

- **Trusting Client-SideData**:do not trust client-sidedata (Identify all input parameters that trust client-side data). **Unescaped Special Characters**: Unescaped special
- characters within input strings. **Authentication mechanisms** using technologies such as JavaScript or
- ActiveX. **Lack of re-authenticating** the user before issuing new passwords or performing critical tasks.
- Hosting of uncontrolled data on a **protected domain**.
-



KEY POINTS OF WEB APPLICATION SECURITY

- **Authentication :** Ensuringthatusers are whothey say theyare (e.g., loginsystems, multi-factor authentication).
- **Authorization :** Ensuringthat usershave permission to access specific resources (e.g., role-based access control). **Data**
- **Encryption :** Protecting sensitive data both in transit (using HTTPS) and at rest (using encryption algorithms). **Input**
- **Validation :** Preventing harmful input from users (e.g., SQL injection, cross-site scripting). **Session Management:**
- Securing user sessions (e.g., using secure cookies, session timeouts). **Cross-Site Scripting (XSS) Prevention:**
- Preventing attackers from injecting malicious scripts into web pages. **Cross-Site Request Forgery (CSRF) Prevention** –
- Preventing unauthorized commands from being sent by authenticated users. **Error Handling** – Ensuring errors don't expose sensitive information (e.g., database structure, server paths). **Security Headers** – Adding HTTP security
- headers (e.g., Content Security Policy, Strict-Transport-Security). **Regular Patching and Updates** – Keeping software,
- libraries, and systems up to date with the latest security patches. **Logging and Monitoring –** Tracking and alerting for
- suspicious activity or security events (e.g., failed login attempts). **Access Control** – Limiting access to resources based on user roles or permissions (e.g., least privilege principle). **SQL Injection Prevention** – Using prepared statements or
- ORM to prevent attackers from manipulating database queries. **Security Testing –** Regular vulnerability assessments,
- penetration testing, and code reviews to identify weaknesses. **Two-Factor Authentication (2FA)** – Adding an
- additional layer of security beyond just passwords. **Security Best Practices** – Implementing security frameworks and coding standards (e.g., OWASP guidelines). **Backup and Recovery** – Regularly backing up critical data and having plans
- for recovery in case of a security breach.
-
-
-

IMPORTANCE OF REPORTING IN ETHICAL HACKING

Reporting helps a company understandand fixsecurity issues to make their systems safer. If an ethical hacker discovers a SQL injection vulnerability in a website, they would report:



25 **The issue:** SQL injection, which could let attacker's access sensitive data.

25 **Risk level:** High, because it impacts the database directly.

25 **Steps to fix:** Use prepared statements and sanitize inputs

Vulnerability Remediation:

VulnerabilityRemediationistheprocessoffindingandfixing weaknesses in a system or software to protect it from being attacked.

Vulnerability Remediation Process:

- Identifying Vulnerabilities
- Prioritizing Vulnerabilities
- Fixing the Issues Testing
- and Verification
- Monitoring and Updates

Example:

Fixing an XSS Vulnerability

Problem: A website allows hackers to inject harmful code into input fields (XSS).
Fix: Add input validation to block harmful code. **Test:** Check to make sure the input fields no longer accept malicious code.

ETHICS AND POST-ENGAGEMENT CONSIDERATIONS

Ethics:

Ethical hackers work with authorization, focusing on identifying and fixing security issues without misusing information.

Post-Engagement Considerations:

Aftera security test, ethicalhackers should:
Report Findings: Share a detailed report of discovered issues and solutions. **Remove Tools:** Clear any scripts or tools to prevent future misuse. **Keep Information Private:** Protect sensitive data and avoid sharing it without permission. **Restore Systems:** Return everything to normal, ensuring no disruptions or risks remain.

Identifying and Exploiting SQL Injection

PRACTICAL 1

Purpose:

The goal is to demonstrate how a website can be vulnerable to SQL injection and show how attackers can exploit this weakness to gain unauthorized access. Test a login form by entering admin' OR 1=1 -- in the username field. If it bypasses authentication, it shows the site is vulnerable to SQL injection, allowing attackers to access sensitive information.

Activity:



Basic Cross-Site Scripting (XSS) Attack Demonstration.

PRACTICAL 2

Purpose: To demonstrate how XSS vulnerabilities allow attackers to inject malicious scripts into websites.

Activity: Enter a simple script, like <script>alert('XSS')</script>, into a form. If the script runs, it shows the site is vulnerable to XSS attacks.

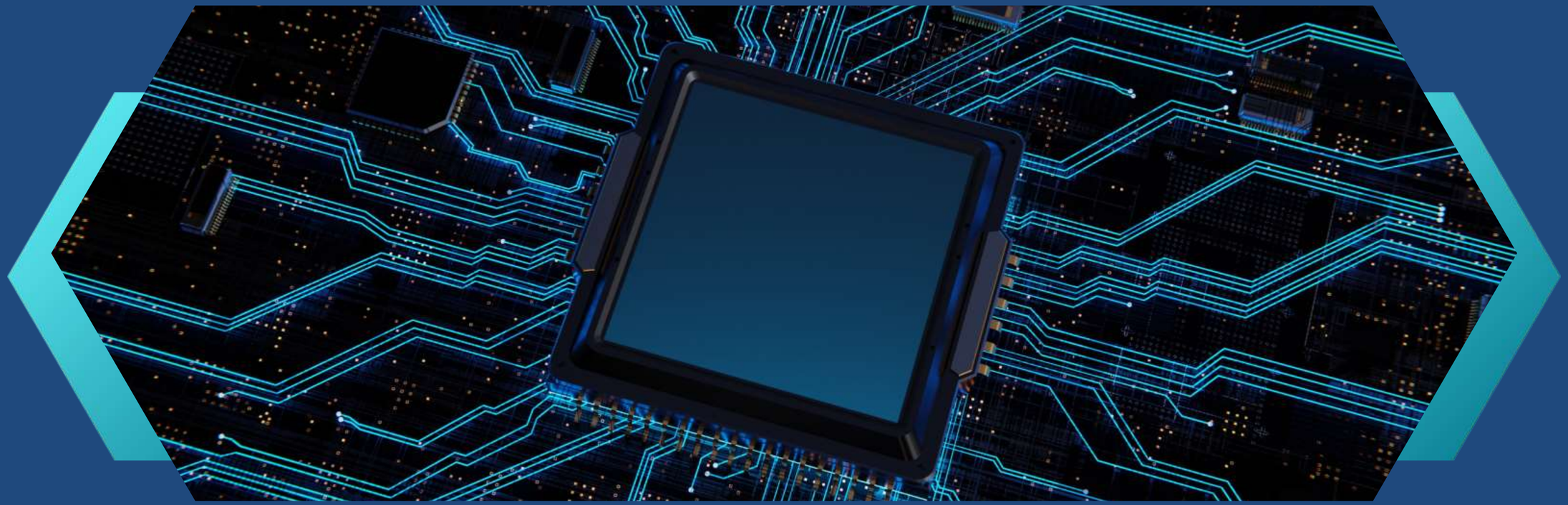


Using Burp Suite for Web Application Testing.

PRACTICAL 3

Purpose: Use Burp Suite to find and fix security weaknesses in a web application.

Activity: Intercept and analyse web traffic with Burp Suite to test for vulnerabilities by modifying requests.



PREPARING A SAMPLE PENTESTING REPORT

A **PenetrationTestingreport**isadocumentthat containsa detailedanalysisofthevulnerabilities uncovered during the security test.

- **Executive Summary:** When the Pentest activity was performed, overview of test purpose and results.
- **Total Vulnerabilities:** How many of them are Critical, High, Medium, Low, Info, Zero-Day and Exploits.
- **Scope:** Details on what was tested and permissions granted. **Methodology:** Steps and tools used during the test. **Findings:** List of vulnerabilities by severity with risk explanations. **Recommendations:** Clear steps to fix issues, prioritized by risk level. **Steps of Finding Vulnerability:** How Vulnerability is found out, how it occurs and where it occurs. **Proof of Concept:** Screenshots or logs to show exploitation. **Conclusion:** Security summary and need for fixes. **Appendices:** Technical details and additional evidence.